W

WANDA

Where Are you Now Detection App

Smart home tracking Android application with an integrated shopping list

Course / Subject	Sistemi ad Agenti
Platform	Android (API 26–34)
Language	Kotlin
Version	1.0
Architecture	MVVM
Authors	Enrique Robeles Juan Felix Noblejas Torroba Alex Rodriguez Bernardo Gómez

Table of Contents

1. Project description
2. WANDA as a reactive agent
3. Core features
4. Project architecture
5. Tech stack
6. Description of key components
7. Database (Room / SQLite)
8. Application screens
9. Required permissions
10. How to build and run
11. Step-by-step user guide
12. Advanced configuration
13. Technical design decisions
14. Troubleshooting

1. Project description

WANDA (*Where Are you Now Detection App*) is an Android application developed in Kotlin for the **Sistemi ad Agenti** course. Its main goal is to practically demonstrate the principles of a reactive agent situated within a real physical environment: the user's home.

The application acts as a **domestic context agent**. It continuously monitors the device's GPS position and makes automated decisions based on whether the user is inside or outside a defined geographical perimeter, known as a *geofence*. Whenever the user crosses this perimeter —either when leaving or arriving home— WANDA reacts immediately: it logs the event, stores it in a local database, and alerts the user via a push notification.

As a complementary feature, the app integrates a **persistent shopping list** that the user can manage from any screen. Although it is not part of the core agent logic, it enriches the app's overall value proposition as a useful household support tool.

The project is structured following the **MVVM** (Model-View-ViewModel) architectural pattern recommended by Google for Android, ensuring a clean separation of concerns, maintainable code, and a smooth, reactive user experience.

2. WANDA as a reactive agent

Within the scope of this course, an **agent** is defined as an entity capable of perceiving its environment through sensors and acting upon it via effectors, autonomously and continuously. WANDA serves as a concrete example of a **sensor-based reactive agent** operating in the user's physical environment.

THE PERCEPTION – DECISION – ACTION CYCLE

> **PERCEPTION** ⟶ Device GPS detects real-time geographical coordinates **DECISION** ⟶ Geofencing API evaluates if the device crossed the boundary ENTER transition? → type = "ENTRADA" (ENTER) EXIT transition? → type = "SALIDA" (EXIT) **ACTION** ⟶ 1. Saves GpsRecord to the local database (Room / SQLite) 2. Issues a push notification to the user 3. Updates the UI in real time via LiveData

The agent is **reactive** because it does not plan ahead: it responds directly to environmental stimuli without maintaining a complex internal world model or executing deliberative reasoning. As soon as the system detects a geofence transition, the response is immediate and deterministic.

AUTONOMY AND PERSISTENCE

A core property of agents is **autonomy**: the ability to operate without continuous user intervention. WANDA implements this through two mechanisms:

- **Foreground Service:** An Android service running in the foreground that stays alive even when the app is closed or the device is idle. The agent never stops tracking as long as the service is enabled.
- **BootReceiver:** If the device is turned off or reboots, the agent automatically reactivates upon startup, restoring the geofence tracking without requiring user action.

ENVIRONMENT, SENSORS, AND EFFECTORS

Agent Component	Implementation in WANDA
Environment	Physical world: user's geographical position
Sensor	Device GPS + Google Play Services Location
Perception Function	Geofencing API: detects zone transitions
Internal State	PreferencesManager (home coordinates, radius, active tracking state)
Effector — Storage	Room DB: stores the event as a GpsRecord

Agent Component	Implementation in WANDA
Effector — Communication	NotificationManager: sends push notifications to the user
Effector — UI	LiveData: updates the home screen in real time

3. Core features

SET HOME LOCATION

Users can define their home location in two ways. First, by clicking "Set Home Here", the app captures the current GPS coordinates and saves them as a reference. Second, users can manually enter the latitude, longitude, and radius through an edit form, which is useful for setting up a location other than where the device currently is.

CONFIGURABLE GEOFENCE

A *geofence* is a virtual circular boundary defined by center coordinates and a radius in meters. WANDA allows users to adjust this radius between **50 and 5,000 meters** (defaults to 100 m). A smaller radius offers high precision but might cause false alerts due to GPS drift, while a larger radius is more forgiving but might trigger before physically arriving home.

BACKGROUND TRACKING

Once tracking is activated, WANDA fires up a *Foreground Service* that keeps the geofence registered within Google Play Services indefinitely. It displays a persistent status bar notification ("WANDA – Tracking active") to comply with Android guidelines and keep the user informed that monitoring is underway.

ENTRY AND EXIT DETECTION

When the device crosses the geofence perimeter, Google Play Services generates a transition event. WANDA intercepts this event and performs three actions simultaneously: it logs a database entry with the precise time and coordinates, sends a push notification to the user, and updates the state on the home screen.

EVENT HISTORY LOG

All triggered events are stored persistently in the local database. The history screen displays each event with its type (ENTRADA / SALIDA), formatted date and time, and the exact GPS coordinates where the crossing occurred. The list updates dynamically via LiveData.

SHOPPING LIST

An auxiliary feature designed to manage household pending items. Each item can be marked as bought with a single tap (applying a visual strikethrough text styling), and a single button allows clearing all completed items at once. The data persists locally.

REBOOT PERSISTENCE

If the device restarts while tracking was active, the `BootReceiver` catches the system's `BOOT_COMPLETED` broadcast and automatically restarts the location service, restoring the tracking seamlessly.

Feature	Description
Set Home Location	Automated GPS detection or manual coordinate entry
Configurable Geofence	Adjustable radius ranging from 50 to 5,000 meters
Background Tracking	Foreground Service remains active even when the app is killed
Entry/Exit Detection	Automated push notification when crossing boundaries
Event History	Persistent log containing timestamps and precise coordinates
Shopping List	Interactive item list with toggleable completion states
Reboot Persistence	Agent automatically restores monitoring upon device boot

4. Project architecture

WANDA follows the **MVVM** (Model-View-ViewModel) architectural pattern, which is the official standard recommended by Google for modern Android applications. This architecture separates the app into three distinct layers with clean boundaries.

MODEL — DATA

Manages data storage and business data access logic. Contains Room database, DAOs, entities, and repositories. Completely independent of the UI layer.

VIEWMODEL — DOMAIN

Handles business logic and prepares data for the UI. Exposes states via LiveData and survives configuration changes like screen rotations.

VIEW — PRESENTATION

Fragments and XML layouts. Only observes UI states from the ViewModel. Never accesses the database or GPS APIs directly.

DIRECTORY STRUCTURE

```
com.wanda.app/
├── MainActivity.kt               # Main activity + Bottom Navigation Setup
├── WandaApplication.kt           # Initializes DB, repositories, and notification c
│
├── data/                         # DATA LAYER
│   ├── database/
│   │   ├── WandaDatabase.kt      # Room Database (SQLite abstraction)
│   │   └── dao/
│   │       ├── ShoppingDao.kt    # CRUD for shopping list items
│   │       └── GpsRecordDao.kt   # CRUD for GPS event history
│   ├── entity/
│   │   ├── ShoppingItem.kt       # Entity: shopping list item
│   │   └── GpsRecord.kt          # Entity: GPS geofence event record
│   └── repository/
│       ├── ShoppingRepository.kt
│       └── GpsRepository.kt
│
├── geofencing/                   # TRACKING & GEOFENCING CORE
│   ├── GeofenceHelper.kt         # Manages Google Geofencing Client API wrappers
│   ├── GeofenceBroadcastReceiver.kt  # Handles incoming geofence transition alerts
│   ├── LocationTrackingService.kt    # Foreground service handling active tracking
│   └── BootReceiver.kt           # Restores tracking service on system boot
│
├── ui/                           # PRESENTATION LAYER (MVVM)
│   ├── home/     → HomeFragment + HomeViewModel
│   ├── shopping/ → ShoppingFragment + ShoppingViewModel + ShoppingAdapter
│   └── history/  → HistoryFragment + HistoryViewModel + HistoryAdapter
│
└── utils/
    ├── PreferencesManager.kt     # SharedPreferences configuration wrapper
    └── PermissionHelper.kt       # Validates and requests runtime permissions
```

DATA FLOW

1. The user configures home parameters in HomeFragment → changes are saved in PreferencesManager.
2. When enabling tracking, HomeFragment starts LocationTrackingService.
3. LocationTrackingService commands GeofenceHelper to register the geofence with Google Play Services.
4. When a crossing occurs, Google notifies GeofenceBroadcastReceiver.
5. The receiver writes a new GpsRecord into Room (via GpsRepository) and sends a push notification.
6. HistoryFragment and HomeFragment observe the DB updates via LiveData, updating the UI instantly.

5. Tech stack

Technology	Version	Purpose in the Project
Kotlin	1.x	Core language. Concise, safe, with built-in native support for coroutines.
Android SDK	API 26–34	Target platform. API 26 minimum ensures wide device compatibility.
AndroidX Navigation	2.7.7	Manages Fragment transactions connected to the Bottom Navigation Bar.
Room (SQLite)	2.6.1	Official Android ORM. Abstraits SQLite with compile-time query validation.
Google Play Services Location	21.2.0	Provides FusedLocationProvider (battery-efficient GPS) and the Geofencing API.
ViewModel + LiveData	2.7.0	Backbone of the MVVM implementation. LiveData is lifecycle-aware and observable.
Kotlin Coroutines	1.7.3	Asynchronous concurrency without callback hell. Used for database I/O tasks.
ViewBinding	—	Type-safe and null-safe access to XML views. Completely replaces `findViewById`.
KSP	—	Kotlin Symbol Processing annotation tool for Room, speeding up build times over KAPT.
Material Design	1.11.0	Standardized UI components (buttons, dialogs, FAB, themes) per Google guidelines.

6. Description of key components

WANDAAPPLICATION

A custom `Application` class that handles manual dependency injection container initialization. It boots up prior to any Activity, instantiating the Room database, repositories, `PreferencesManager`, and required Android 8+ notification channels. By exposing these singletons globally, components can easily pull them without adding third-party DI frameworks.

PREFERENCESMANAGER

A thin wrapper over `SharedPreferences` encapsulating persistent configuration metrics for the reactive agent. It stores four core key-value flags:

- `homeLatitude` / `homeLongitude`: Decimal values representing the home location.
- `geofenceRadius`: Boundary distance threshold in meters (defaults to 100 m).
- `isTrackingEnabled`: Flag checking whether tracking is turned on; used by `BootReceiver`.
- `isHomeSet`: Controls if home is ready. Until set, tracking buttons are locked.

LOCATIONTRACKINGSERVICE

An Android Service declaring `foregroundServiceType="location"` that drives the core of the agent. Upon launch, it places a persistent tray notification (mandatory for modern location collection), grabs settings from `PreferencesManager`, and uses `GeofenceHelper` to pass the boundary data to Google Play Services. It returns `START_STICKY` to restart automatically if killed due to low memory.

GEOFENCEHELPER

Handles interactions with the `GeofencingClient`. It drafts circular boundaries with adjustable radiuses and assigns a `NEVER_EXPIRE` lifespan. It tracks both `GEOFENCE_TRANSITION_ENTER` and `GEOFENCE_TRANSITION_EXIT` events, routing data to a `PendingIntent` pointing directly to the `GeofenceBroadcastReceiver`.

> **Android 12+:** The geofencing PendingIntent must attach an explicit action intent named `com.wanda.app.ACTION_GEOFENCE_EVENT`, and the receiver definition must flag `exported="true"` in the AndroidManifest, because Google Play Services pushes the broadcast from outside our application sandbox. If omitted, the receiver will never capture events.

GEOFENCEBROADCASTRECEIVER

A broadcast target triggered on geofence entry/exit. It calls `goAsync()` to secure extra computing execution time past the default ~10-second ceiling, allowing the async database task to finalize safely. It reads transition flags, submits a `GpsRecord` using an IO thread dispatcher, fires a user notification, and wraps up execution with `pendingResult.finish()`.

BOOTRECEIVER

Listens for the system's `BOOT_COMPLETED` intent. On cold start, it reads the `isTrackingEnabled` flag within `PreferencesManager` and, if true, launches `LocationTrackingService` to set up the boundaries again without user intervention.

7. Database (Room / SQLite)

WANDA utilizes **Room** as an abstraction toolkit above local SQLite setups. Room provides compile-time verification for queries (catching errors during compilation rather than crashing at runtime) and links seamlessly with Coroutines and LiveData. The database schema holds two separate tables:

SHOPPINGITEM — TABLE `SHOPPING_ITEMS`

```
@Entity(tableName = "shopping_items")
data class ShoppingItem(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val isChecked: Boolean = false,      // marked as purchased
    val createdAt: Long = System.currentTimeMillis()
)
```

GPSRECORD — TABLE `GPS_RECORDS`

```
@Entity(tableName = "gps_records")
data class GpsRecord(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val type: String,                    // "ENTRADA" (ENTER) or "SALIDA" (EXIT)
    val timestamp: Long = System.currentTimeMillis(),
    val latitude: Double,
    val longitude: Double
)
```

AVAILABLE DATABASE OPERATIONS (DAOS)

DAO	Method Name	Description
GpsRecordDao	`getAllRecords()`	LiveData stream returning all events, sorted descending by timestamp
GpsRecordDao	`insertRecord(record)`	Saves a geofence crossing event (suspend)
GpsRecordDao	`getLastRecord()`	Fetches the latest record entry to show context statuses on Home
ShoppingDao	`getAllItems()`	LiveData stream returning all items sorted by creation time
ShoppingDao	`insertItem(item)`	Inserts a new product into the shopping list (suspend)
ShoppingDao	`updateItem(item)`	Updates checked state variations (suspend)

DAO	Method Name	Description
ShoppingDao	`deleteCheckedItems()`	Clears out all checked products from the list (suspend)

8. Application screens

Application flows are unified via a **Bottom Navigation Bar** connecting three key layout destinations managed by Jetpack's Navigation component.

HOME DASHBOARD (HOMEFRAGMENT)

The main dashboard for the agent. It displays the user's real-time calculated context status ("You are home" / "You left home"), the last documented boundary log, and ongoing configuration tracking maps. From here, the user can:

- **Set Home Here:** Locks current device GPS coordinates as the primary home address.
- **Edit Home:** Modifies values manually (Latitude, Longitude, and a 50–5,000m radius). If tracking is running, the underlying service automatically updates.
- **Clear Home:** Deletes reference coordinates and completely terminates active tracking.
- **Start / Stop Tracking:** Controls the `LocationTrackingService` state. It is enabled only when home coordinates are present.

SHOPPING BASKET (SHOPPINGFRAGMENT)

Manages grocery needs using a reactive `RecyclerView` interface. Checked products show a clear text-strike effect. Key UI buttons: a floating FAB button to append products, tap interactions to check off listings, and a "Clear Completed" shortcut (visible only when checked elements are present).

LOG HISTORY (HISTORYFRAGMENT)

A chronological history feed listing all recorded geofence crossing logs. Each card item specifies transition types (ENTRADA / SALIDA), clean date formatting, and the GPS markers. Emits a placeholder text if empty. The stream updates automatically via LiveData mapping.

9. Required permissions

Permission String	Requirement Group	Justification Purpose
`ACCESS_FINE_LOCATION`	Runtime Prompt	Full GPS fine location metrics crucial for close metric evaluation.
`ACCESS_COARSE_LOCATION`	Runtime Prompt	Approximated location mesh fallback if fine GPS locks take too long.
`ACCESS_BACKGROUND_LOCATION`	Special System Menu	Background location access (Android 10+). Imperative for background tracking.
`FOREGROUND_SERVICE`	Normal Manifest	Allows launching foreground location tracking workflows securely.
`FOREGROUND_SERVICE_LOCATION`	Normal Manifest	Declares explicit foreground-level intent to scan coordinate systems (Android 14+).
`POST_NOTIFICATIONS`	Runtime Prompt	Required to show alerts and the sticky background tracking notification (Android 13+).
`RECEIVE_BOOT_COMPLETED`	Normal Manifest	Listens for device startups to re-enable automated agents.

> **Important Configuration:** `ACCESS_BACKGROUND_LOCATION` must be enabled manually by the user: Settings → Apps → WANDA → Permissions → Location → select *"Allow all the time"*. Android security blocks bundling this inside multi-permission prompts. Without it, tracking stops immediately when minimizing the app.

10. How to build and run

PREREQUISITES

- Android Studio Hedgehog (2023.1.1) or newer
- JDK 17 (pre-bundled with modern Android Studio variations)
- Android SDK support matching API level 26 or above
- A physical Android testing phone with Google Play Services installed

COMPILATION STEPS

1. Open Android Studio, select **File** → **Open**, navigate to `[path]/Ad-agenti/WANDA/`, and pick the folder as root project.
2. Let Gradle sync up by selecting **File** → **Sync Project with Gradle Files**. Ensure you have an internet connection to pull initial dependencies.
3. Connect your physical testing phone via USB with **USB Debugging enabled** under Developer Options.
4. Pick your active phone name inside the taskbar device menu, then click the green **Run** button (▶) or press `Shift+F10`.

> **Warning Note:** Google Play Services Geofencing setups **do not fire reliably on virtual Emulators**. Testing requires hardware telemetry and real-time network access. A physical device is required to validate geofencing actions.

11. Step-by-step user guide

INITIAL ONBOARDING

1. Launch WANDA. App prompts will ask for location tracking permissions. Approve each one.
2. On devices with Android 10+, navigate manually to Settings → Apps → WANDA → Permissions → Location and choose **"Allow all the time"**.
3. While standing inside your house, press **"Set Home Here"**. Coordinate rows and the radius configuration (100 m default) will instantly lock in.
4. If needed, press **"Edit Home"** to adjust the perimeter radius to fit your street layout.
5. Tap **"Start Tracking"**. The sticky notification stating "WANDA – Tracking active" will display. The agent is now fully running.

DAILY USAGE

- Stepping outside boundaries: A "You left home" alert arrives and a history row is saved.
- Returning inside boundaries: A "You arrived home" card pushes, updating the UI to state "You are home".
- Tracking persists across device locks, app swiping tasks, or unexpected reboots.
- To switch the sensor off entirely, enter WANDA dashboard and tap "Stop Tracking".

USING THE SHOPPING LIST

- Select the shopping cart button choice on the bottom nav tab link.
- Press the floating action item **"+"** button to name and append items.
- Tap on any entry to stamp it bought (striking text visuals apply).
- Tap **"Clear Completed"** to remove checked listings off the board.

12. Advanced configuration

ADJUSTING GEOFENCE RADIUS

The radius parameter dictates how far away from the house tracking registers actions. Reference targets:

- **50–100 m:** Great for isolated homes or precise close-range evaluation.
- **150–300 m:** Recommended choice for high-density metropolitan spots to handle cellular/Wi-Fi drift deviations.
- **500–5,000 m:** Used to track broader macro-spaces like urban neighborhoods or workplace zones.

To configure: Tap **"Edit Home"** on the main dashboard. If background services are active, they will safely reboot to adapt to the new perimeter value.

RELOCATING HOME CENTERPOINTS

- **Option A:** Choose "Edit Home" and write structural numeric double markers in classic format choices (e.g., 40.416775, -3.703790).
- **Option B:** Tap "Clear Home", move to your new location, and tap "Set Home Here".

13. Technical design decisions

WHY USE A FOREGROUND SERVICE?

Android memory cleanups are highly aggressive and terminate background tasks to free up RAM. Standard background services get terminated within minutes. A **Foreground Service**, anchored by a visible status icon, gets designated with higher process lifecycle priority and is rarely killed. The trade-off (the mandatory sticky notification) serves perfectly to remind the user that context tracking is active.

WHY CALL `GOASYNC()` IN THE BROADCASTRECEIVER?

Android limits `BroadcastReceiver` executions to a strict **10-second** budget. Accessing and saving entries into Room database targets involves disk I/O, which could lag on low-end devices. Calling `goAsync()` shifts processing lifespans safely until `pendingResult.finish()` is executed, avoiding Application Not Responding (ANR) flags.

WHY ATTACH EXPLICIT ACTIONS ON THE PENDINGINTENT?

Starting with Android 12 (API 31), implicit intent receivers are restricted for security reasons. Because the Geofencing system sends updates from external system components, we must set an explicit action identifier (`com.wanda.app.ACTION_GEOFENCE_EVENT`) and expose the target using `exported="true"` in the manifest. If missing, transitions will fail to fire inside our app.

WHY INTRODUCE A BOOTRECEIVER?

Active geofence parameters registered with Google Play Services **are wiped completely** when a device restarts. Without `BootReceiver` , tracking would die silently after a phone restart. Capturing `BOOT_COMPLETED` allows reading persistent preferences and automatically starting services back up seamlessly.

WHY CHOOSE LIVEDATA OVER KOTLIN FLOWS?

LiveData was selected because of its built-in synergy with Fragment lifecycles. It stops sending streams as soon as views transition to background states, preventing memory leaks or crashes from view references being destroyed. Given the scale of this project, LiveData delivers sufficient power with lower complexity than Coroutine Flow pipelines.

14. Troubleshooting

Encountered Error	Probable Cause	Resolution Step
No notifications arrive on entry/exit	Background location access missing	Settings → Apps → WANDA → Permissions → Location → Choose "Allow all the time".
Alert signals take too long to push	GPS signals drift or radius bounds are too small	Increase radius parameters to 150–300 m. Note that Geofencing engines can take 2–5 minutes to evaluate transitions.
Tracking stops working after a phone reboot	BOOT_COMPLETED is blocked or battery optimization is killing the process	Disable battery saving rules: Settings → Battery → Optimization → WANDA → Select "Don't optimize".
Geofences never trigger updates	Google Play Services is out of date	Update Google Play Services framework apps inside the Google Play Store.
Errors when locking location hooks via GPS	GPS features disabled or inside concrete structures	Turn location configuration to "High Accuracy". Walk outside to catch clear satellite telemetry.
The application fails inside an emulator	Geofencing mechanics demand physical hardware radios	Deploy and build the project exclusively on physical Android phones.
Sticky service notifications fail to show up	POST_NOTIFICATIONS access denied	Enable notifications: Settings → Apps → WANDA → Notifications → Toggle on.
The tracking service stops randomly	Aggressive OEM task killer restrictions	On Xiaomi, Huawei, or Samsung variants, add WANDA to unmonitored background exception checklists.

> **Manufacturer Note:** Brands running custom Android flavors (Xiaomi MIUI/HyperOS, Huawei EMUI, Samsung One UI) deploy rigid task management engines that can interfere with background services. Review *dontkillmyapp.com* for specific device handling workarounds.

WANDA v1.0 · Sistemi ad Agenti · Android (API 26–34) · Kotlin + MVVM

Enrique Robeles · Juan Felix Noblejas Torroba · Alex Rodriguez · Bernardo Gómez